

Effect of Associativity on Cache Performance

Implementation and experimental evaluation of a single-level set-associative data-cache simulator

Prepared by: Project Team

Configuration: 64 KB cache · 64 B lines · Random replacement

Submission date: July 19, 2026

1. Cache Implementation and Verification

The project implements a real set-associative cache while keeping the provided memory generators and experiment harness unchanged. The simulator models tags, valid bits, dirty bits, random replacement, and a write-back/write-allocate policy.

1.1 Data organization and address mapping

The cache is stored as a two-dimensional `vector<vector<CacheLine>>`. The outer dimension represents sets, and each inner vector contains the ways in that set. Every cache line stores a tag, a valid bit, and a dirty bit. At the start of each run, `initCache(ways)` recalculates the number of sets, resizes the structure, clears all valid and dirty bits, resets the write-back count, and resets the replacement random-number stream.

$$\text{number of sets} = 64 \text{ KB} \div (64 \text{ B} \times \text{ways})$$

Address parsing uses arithmetic rather than compiler-specific bit functions. The block number is `address / LINE_SIZE`; the offset is the remainder within the 64-byte line; the set index is `block_number % num_sets`; and the remaining upper value is the tag. This method produces the required `tag | set index | block offset` fields for every tested associativity and works with both GCC and Microsoft C++ compilers.

Hit path

The simulator searches every way in the selected set for a valid line with a matching tag. A matching read returns a hit. A matching write also sets the dirty bit because the updated value now exists only in the cache.

Miss path

An invalid way is used before any valid line is replaced. If the set is full, `randReplace() % ways` chooses the victim. A dirty victim produces one write-back before the requested block is allocated.

1.2 Write policy

The implementation follows write-back with write-allocate. Every read or write miss loads the requested block into the cache. A write miss installs a dirty line, while a read miss installs a clean line. Memory traffic from replacement is counted only when a valid dirty line is evicted. This separates cache hits from the additional cost of writing modified data back to DRAM.

1.3 Evidence of testing

The final source was compiled with GCC using C++17 and strict warnings (`-Wall -Wextra -Wpedantic -Wconversion`). An automated CTest regression test then ran the complete experiment again, checked for exactly 30 data rows, and compared the newly generated CSV byte-for-byte against the submitted `results.csv`.

PASS — 1/1 regression test passed; 0 tests failed. The fresh output matched all recorded hit ratios, miss ratios, read/write counts, and write-back counts exactly.

The results also provide independent behavior checks: memGen1 changes from continuous conflict misses at one way to two cold misses at two or more ways, while memGen5 gives the theoretical sequential-stream result of 15 hits per 16 word accesses, or 93.75%.

2. Experimental Setup and Results

64 KB

CACHE CAPACITY

1,000,000

ACCESSES PER RUN

30

TOTAL RUNS

The experiment holds cache size, line size, replacement policy, write policy, and address space fixed. Only associativity changes. Each of the six unmodified generators is tested at 1, 2, 4, 8, and 16 ways from a cold cache. The address space is 64 MB, the line size is 64 B, replacement is random, and writes use write-back with write-allocate.

2.1 Hit-ratio table

Generator	Access pattern	1-way	2-way	4-way	8-way	16-way
memGen1	K=2 same-set ping-pong	0.0000%	99.9998%	99.9998%	99.9998%	99.9998%
memGen2	Random over 64 MB DRAM	0.0972%	0.0996%	0.0983%	0.0969%	0.0983%
memGen3	K=8 same-set cyclic	0.0000%	0.7835%	17.6736%	99.9992%	99.9992%
memGen4	16 sets $\times$ K=4 conflict	0.0000%	14.2802%	99.9936%	99.9936%	99.9936%
memGen5	Sequential word stream	93.7500%	93.7500%	93.7500%	93.7500%	93.7500%
memGen6	Stack push/pop	99.9921%	99.9921%	99.9921%	99.9921%	99.9921%

Table 1. Measured hit ratio for every generator and associativity. Values come directly from results.csv.

2.2 Hit ratio versus number of ways

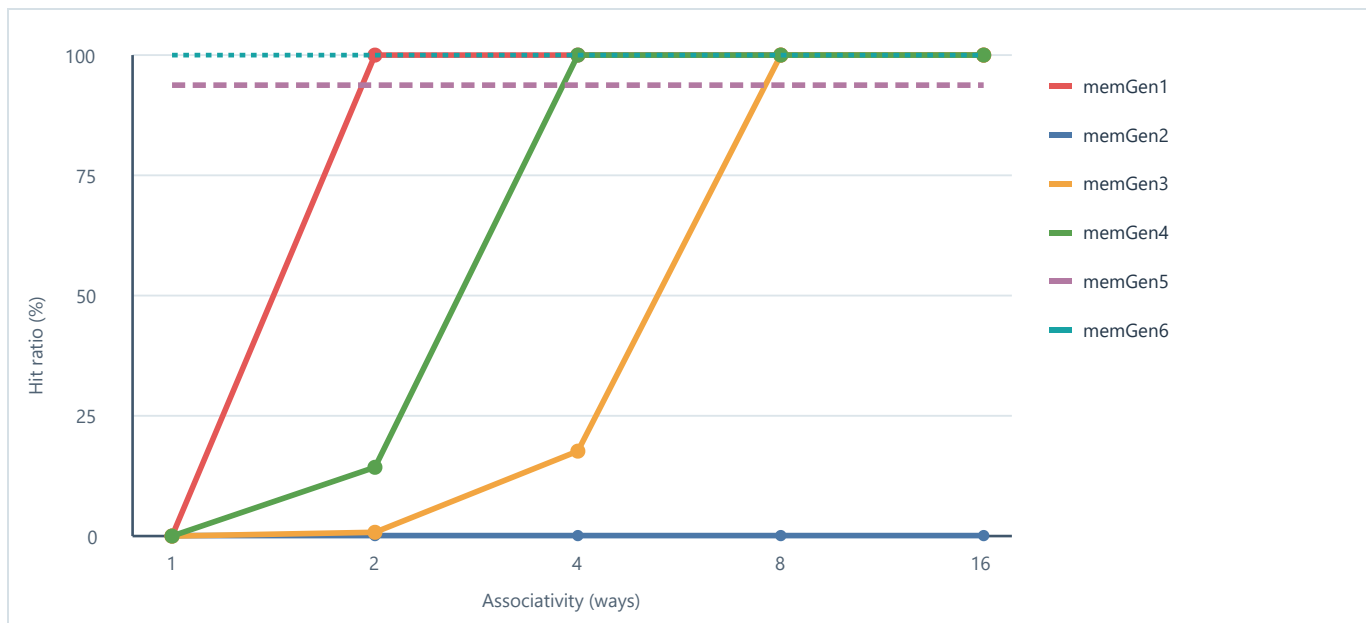


Figure 1. Conflict patterns rise when enough ways are available; locality and capacity patterns remain nearly flat.

3. Analysis of Access Patterns

3.1 Conflict-sensitive patterns

memGen1 alternates between two blocks that map to the same set. A direct-mapped cache can keep only one of them, so each access removes the block needed next and the hit ratio is 0%. At two ways, both blocks remain in the set. Only the first access to each block misses, producing 99.9998% hits. More than two ways gives no further benefit.

memGen3 repeats the same situation with eight conflicting blocks. With 1, 2, or 4 ways, the set is too small and random replacement often removes a block before its next use. This explains the gradual rise from 0% to 0.7835% and then 17.6736%. At eight ways, the entire group fits, so the hit ratio jumps to 99.9992%. Sixteen ways cannot improve it because the pattern uses only eight lines.

memGen4 applies a four-block conflict pattern across 16 different sets. One way gives 0% hits and two ways gives 14.2802%. At four ways, each set can hold its complete four-line group, and the hit ratio reaches 99.9936%. The number of active sets increases the amount of traffic but does not change the four-way threshold.

Main conflict result: associativity removes thrashing when the number of ways reaches the number of simultaneously active blocks mapped to one set: $K=2$ for memGen1, $K=4$ for memGen4, and $K=8$ for memGen3.

3.2 Patterns that are almost independent of associativity

memGen2 selects addresses randomly across 64 MB, which is about 1,024 times larger than the 64 KB cache. Reuse before eviction is therefore rare. Increasing associativity reorganizes the same fixed capacity; it does not increase the total number of cache lines. The hit ratio remains close to 0.1% for every number of ways. These misses are mainly capacity and compulsory misses, not conflicts.

memGen5 accesses consecutive four-byte words. One 64-byte cache line contains 16 words. The first word in each line misses, while the next 15 words hit, giving a theoretical ratio of $15/16 = 93.75\%$. Because the stream does not return to old lines, extra associativity has no opportunity to help. The exact 93.75% result at all five settings is also a strong check of the line-size and address-mapping logic.

memGen6 models stack pushes and pops in a working region of at most 16 KB. This active data is far smaller than the 64 KB cache and is accessed repeatedly. It therefore has strong temporal locality and remains at 99.9921% for every associativity. The small number of misses comes from initially bringing new lines into the cold cache.

3.3 Capacity versus spatial locality

Capacity limitation

memGen2 needs a much larger effective storage space or more temporal reuse. More ways cannot correct a working set that is much larger than the full cache.

Spatial locality

memGen5 benefits from the 64-byte line itself. Its hits come from nearby words already loaded by the first access, not from keeping several conflicting lines.

4. Diminishing Returns and Conclusions

4.1 Where diminishing returns appear

The results show that more associativity is useful only while conflict misses are still being removed. memGen1 reaches its useful limit at two ways, memGen4 at four ways, and memGen3 at eight ways. After these points, the lines in Figure 1 become flat. The working group already fits inside its set, so additional ways do not create new hits.

This matters because higher associativity is not free in a real processor. More ways require more tag comparisons, a wider selection circuit, and usually greater access energy. They can also increase design complexity and access time. A practical cache should therefore provide enough associativity for common same-set conflicts, but not assume that the largest possible number of ways is always best.

4.2 Summary by generator

Generator	Main behavior	Useful associativity	Reason
memGen1	Two-line conflict	2-way	Both same-set lines fit together
memGen2	Random/capacity misses	No clear benefit	Working set is much larger than cache
memGen3	Eight-line conflict	8-way	All eight same-set lines fit
memGen4	Four-line conflicts in 16 sets	4-way	Each set holds its four-line group
memGen5	Sequential spatial locality	No clear benefit	15 of 16 words hit within each line
memGen6	Small stack working set	No clear benefit	Active data already fits comfortably

4.3 Final conclusion

The simulator produces the behavior expected from a correct set-associative cache. Associativity strongly improves workloads dominated by conflicts, but only until each set can hold the active group of competing blocks. It does not solve a lack of total capacity, and it does not improve patterns whose performance already comes from spatial or temporal locality.

Across the complete sweep, the clearest lesson is that associativity is a targeted solution. Two, four, and eight ways are the correct turning points for the K=2, K=4, and K=8 conflict generators. Above those points, hit ratio remains unchanged. The sequential stream stays at its theoretical 93.75%, the stack stays close to 100%, and random DRAM access stays close to 0.1%. These stable results, together with the automated regression test, support both the correctness of the implementation and the experimental conclusions.

Overall takeaway: increase associativity to control conflict misses, but stop when the workload's same-set working group already fits. Beyond that point, hardware cost rises while cache performance does not.

Reproducibility note

The simulator resets the memory-generator state, cache state, and replacement random-number state before every run. The repository includes the source, CMake build configuration, shared run configuration, automated regression test, testing evidence, and results.csv. Running the program without arguments reproduces the full 30-run experiment and writes results.csv.